

actividad4

March 25, 2025

1 Reconocimiento de Patrones

Objetivos: Implementar los pasos para reconocimiento de patrones de figuras basicas, no se aplica inteligencia artificial.

Nombre: Manuel Ramírez Galván

Fecha: 24/03/25

Actividad: 4

1.1 Ejercicio 1: Importar bibliotecas necesarias

```
[2]: import cv2
import numpy as np
```

1.2 Ejercicio 2: Preprocesamiento de la imagen

Objetivo: Preprocesar una imagen facilitar el reconocimiento de patrones.

Pasos: 1. Cargar una imagen desde disco usando `cv2.imread()`. 2. Convertir la imagen a escala de grises con `cv2.cvtColor()`. 3. Aplicar un umbral usando `cv2.threshold()`. 4. Visualizar la imagen binarizada con `cv2.imshow()`.

```
[ ]: img1_p = cv2.imread('data/figuras.png')
img1 = cv2.cvtColor(img1_p, cv2.COLOR_BGR2GRAY)

_, thresholded_img = cv2.threshold(img1, 127, 255, cv2.THRESH_BINARY_INV)

cv2.imshow('Imagen Original', thresholded_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.3 Ejercicio 3: Identificación de contornos

Objetivo: Detectar contornos para identificar los diferentes objetos.

Pasos: 1. Detectar los contornos de la imagen binaria usando `cv2.findContours()`. 2. Dibujar los contornos detectados con `cv2.drawContours()`. 3. Visualizar la imagen con los contornos resaltados usando `cv2.imshow()`.

```
[ ]: contours, _ = cv2.findContours(thresholded_img, cv2.RETR_TREE, cv2.
    ↳CHAIN_APPROX_SIMPLE)
contour_image = cv2.cvtColor(thresholded_img, cv2.COLOR_GRAY2BGR)

contour_image = img1_p.copy()
cv2.drawContours(contour_image, contours, -1, (255, 255, 0), 2)

cv2.imshow("Imagen con contornos", contour_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.4 Ejercicio 4: Identificación de patrones

Objetivo: Identificar la formas geométrica de figuras en una imagen.

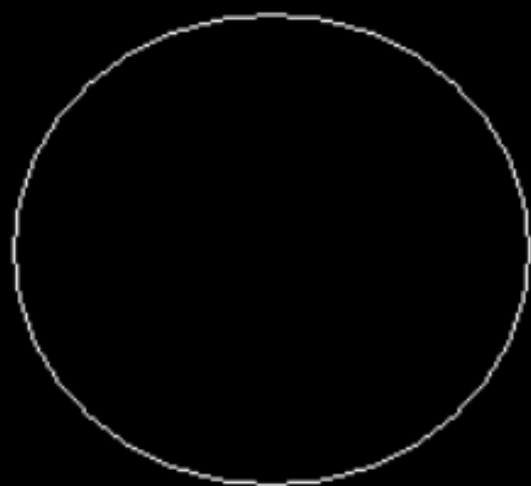
Pasos: 1. Recorrer cada contorno detectado previamente con un ciclo `for`. 2. Define la precisión de la aproximación con `cv2.arcLength()`. 3. Calcular una aproximación del contorno usando `cv2.approxPolyDP()`. 4. Obtenen las medidas de `x,y, w y h` con `cv2.boundingRect()`. 5. Clasificar la figura según la cantidad de vértices obtenidos en la aproximación. 6. Coloca el nombre de la forma sobre la imagen con `cv2.putText()`. 7. Identifica cada figura con un rectángulo usando `cv2.rectangle()`. 8. Visualizar la imagen con las etiquetas y resaltando cada figura con `cv2.imshow()`.

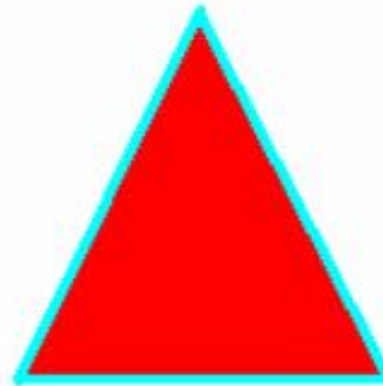
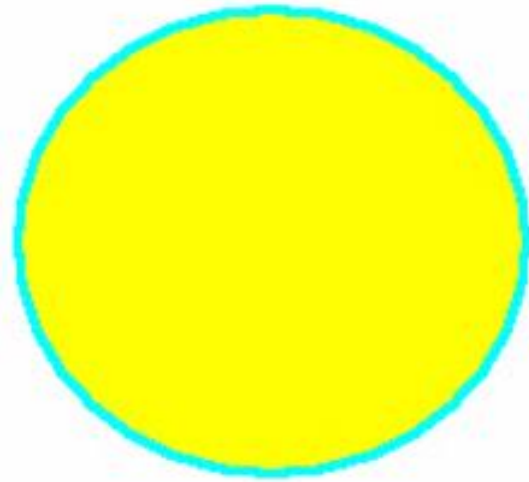
```
[ ]: for contours in contours:
    precision = 0.02*cv2.arcLength(contours, True)
    approximation = cv2.approxPolyDP(contours, precision, True)
    x, y, w, h = cv2.boundingRect(approximation)

    lados = len(approximation)

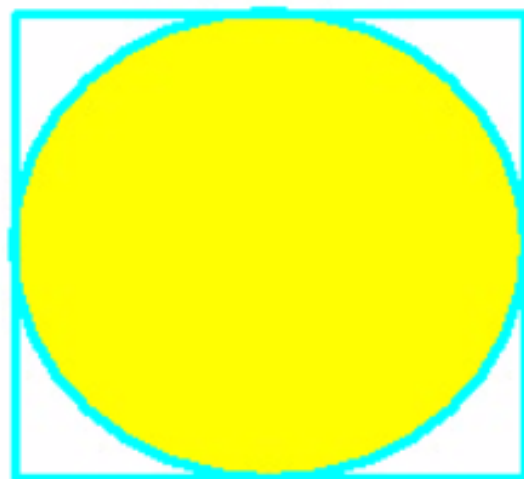
    if lados == 3:
        forma = "Triangulo"
    elif lados == 4:
        forma = "Cuadrado" if abs(w-h) <= 10 else "Rectangulo"
    elif lados > 4:
        forma = "Circulo"
    else:
        forma = "Indefinida"

    cv2.putText(contour_image, forma, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
    ↳ (0, 0, 0), 2)
    cv2.rectangle(contour_image, (x, y), (x+w, y+h), (255, 255, 0), 2)
cv2.imshow("Imagen con contornos", contour_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```





Circulo



Rectangulo



Rectangulo



Triangulo

